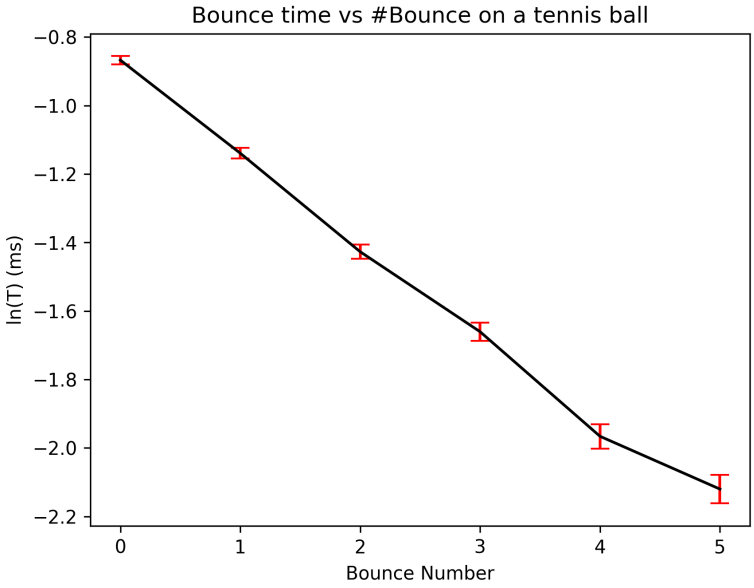
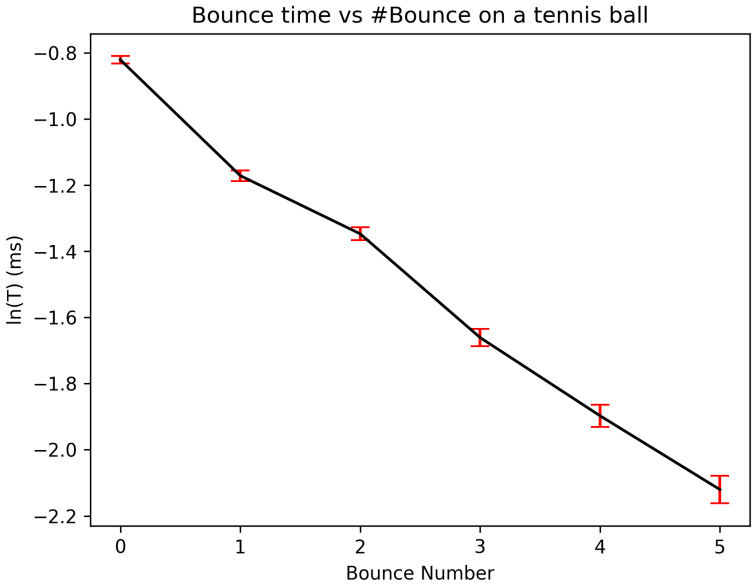
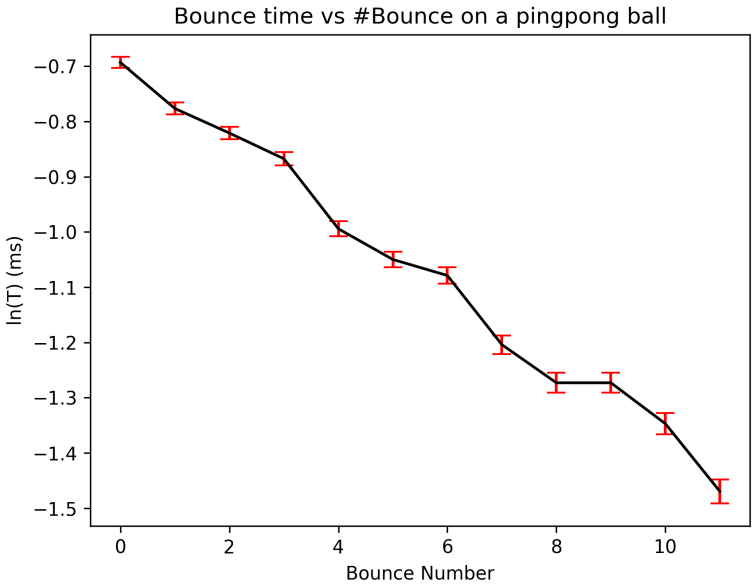
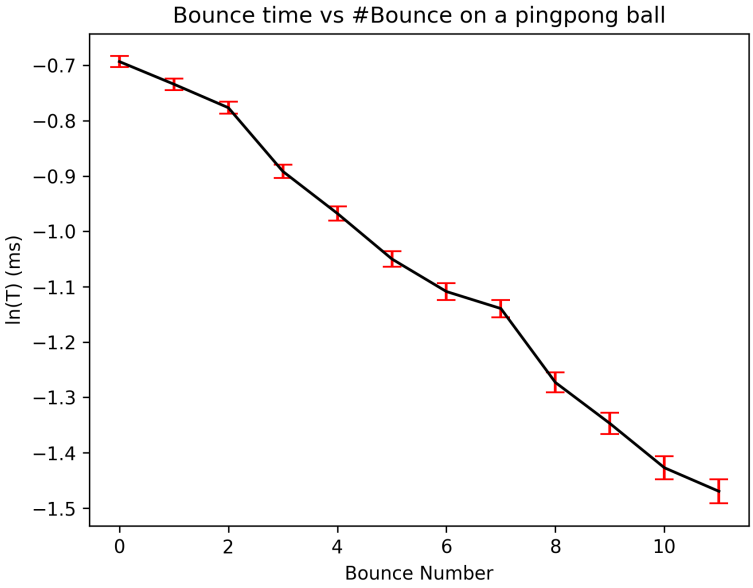
- Part 1
  - All of the values were recorded in a python list
  - `[0.44+/-0.005, 0.31+/-0.005, 0.26+/-0.005, 0.19+/-0.005, 0.15+/-0.005, 0.12+/-0.005]`
    `[0.42+/-0.005, 0.32+/-0.005, 0.24+/-0.005, 0.19+/-0.005, 0.14+/-0.005, 0.12+/-0.005]`
    `[0.5+/-0.005, 0.46+/-0.005, 0.44+/-0.005, 0.42+/-0.005, 0.37+/-0.005, 0.35+/-0.005, 0.34+/-0.005, 0.3+/-0.005, 0.28+/-0.005, 0.28+/-0.005, 0.26+/-0.005, 0.23+/-0.005]`
    `[0.5+/-0.005, 0.48+/-0.005, 0.46+/-0.005, 0.41+/-0.005, 0.38+/-0.005, 0.35+/-0.005, 0.33+/-0.005, 0.32+/-0.005, 0.28+/-0.005, 0.26+/-0.005, 0.24+/-0.005, 0.23+/-0.005]`
  - this is the list of values recorded, the top list is the time difference between the bounces. The uncertainty is 0.005, which is 5ms, which is the distance between each point of the cursor measurement on the oscilloscope.
  - The units of the time recorded are are Seconds.



- Part [[2]]


Bounce time vs #Bounce on a tennis ball
ln(T) (ms)
−0.8
−1.0
−1.2
−1.4
−1.6
−1.8
−2.0
−2.2
0
1
2
3
4
5
Bounce Number


Bounce time vs #Bounce on a pingpong ball
ln(T) (ms)
−0.7
−0.8
−0.9
−1.0
−1.1
−1.2
−1.3
−1.4
−1.5
0
2
4
6
8
10
Bounce Number



– Uncertainty is 5 ms, since the cursor measured in differences of 10ms.
– This part was done in Python as well.
–

```
x = np.array([i + 1 for i in range(len(list))])
y = np.array([math.log(b.n) for b in list])
err_y = np.array([5 / x.n for x in list])
print(y)

x1 = sm.add_constant(x)
w = 1.0 / (err_y**2)
model = sm.WLS(y, x1, weights=w)
result = model.fit()

slope = result.params[1]
slope_error = result.bse[1]
slope_real = np.exp(slope)
slope_error_real = np.exp(slope) * slope_error

intercept = result.params[0]
intercept_error = result.bse[0]
intercept_real = np.exp(intercept)
intercept_error_real = np.exp(intercept) * intercept_error

print(f"intercept = {intercept_real:.4f} ± {intercept_error_real:.4f}")
print(f"slope = {slope_real:.4f} ± {slope_error_real:.4f}")
```

```
result.summary()
return [
    ufloat(slope_real, slope_error_real),
    ufloat(intercept_real, intercept_error_real),
]
```

  - This is a snippet inside a function that returns the exponent of the intercept and the exponent of the slope.
  - The reason we need the intercept and exponent is because what our graph represents.
  - using the given equation $T = (2V_0/g)\epsilon^n$
    * We can solve to
    *
    $$\ln t = \ln(2V_0/g) + n \ln \epsilon$$
    * This is similar to the form y = mx + c. where n = x.
    * but this means that the exp of our intercept and the slope is equal to t and . Which is why we're trying to get the exp of the slope and the intercept.
  - This function above is applied to all 4 of the datasets.
  - `def compsigma(a, b):`
    `return abs((a.n - b.n) / ((((a.s**2) + (b.s**2)) ** (1 / 2))))`
  - This function returns the sigma difference between two ufloats in python. This function allows us to find if our values of epsilon are within 3
  - ```
    a = dofunction(tennisbounces1, "tenboun1", "on a tennis ball")
    b = dofunction(tennisbounces2, "tenboun2", "on a tennis ball")
    c = dofunction(pingpongbounces1, "pp1", "on a pingpong ball")
    d = dofunction(pingpongbounces2, "pp2", "on a pingpong ball")

    print(compsigma(a[0], b[0]))
    print(compsigma(c[0], d[0]))
    ```
  - This is the applied function being applied to epsilons of the four lists.
  - The return of the prints is `0.13894205311038943` and `1.715561143822439`
  - Meaning that our tennis balls are within 0.139 of eachother. Which means that they correlate highly.
  - Our ping pong balls are < 3 , meaning its likely that they are related, but they are off by 1.7 . This is likely due to the ping pong ball being more prone to things like air resistance and confounding variables than a heavier tennis ball.
  - but in conclusion yes, our elasticities are comparable and likely related.
- Part 3
  - First calculate the experimental v0
    *
    $$\frac{1}{2}mv^2 = mgh$$
    *
    $$v = \sqrt{2gh}$$
    ```

- `def getv0(x: ufloat):`
  `return 9.81 * (x) / 2`

  `print(ufloat(math.sqrt(2 * 9.81 * 0.4), 0))`
  `print(getv0(a[1]))`
  `print(getv0(b[1]))`
  `print(getv0(c[1]))`
  `print(getv0(d[1]))`
- This is what the rest of the code looks like. At line 4 we print our experimental value of v, using h = 40cm and 9.81 as our gravity number. this returns a value of `2.8014282071829006+/-0`
- The rest of it uses a top function which gets the v0 based on another calculation.
  - $$\ln(2V_0/g) = y_{\text{intercept}}$$
  - $$v_0 = \frac{ge^{y_i}}{2}$$
  - e^{y_i} and e^{y_i} is the second return value of dofunction.
- This gives us 4 values of v_0.
- which are
- `2.76+/-0.09`
  `2.67+/-0.05`
  `2.623+/-0.034`
  `2.710+/-0.035`
- The top two are the tennis balls. and the bottom two are the ping pong values.
- `v0 = (ufloat(math.sqrt(2 * 9.81 * 0.4), 0))`
  `print(compsigma((getv0(a[1])), v0))`
  `print(compsigma((getv0(b[1])), v0))`
  `print(compsigma((getv0(c[1])), v0))`
  `print(compsigma((getv0(d[1])), v0))`
- This returns values of
- `0.45808485951316164`
  `2.8341223249682415`
  `5.23523235599667`
  `2.644607193846695`
- This means aside from our first pingpong [[test]], we achieve $< 3\sigma$. This makes sense that the pingpong [[test]] is the one that is affected the most, the ball could've been affected by even small air currents.

- All the code is below.
  - “‘ import matplotlib
    matplotlib.use(“TkAgg”) import math import statistics
    import matplotlib.pyplot as plt import numpy as np # imports not on the top nooo import pandas as pd import scipy as sp import

statsmodels.api as sm from scipy import stats from scipy.optimize import curve_fit from uncertainties import ufloat
def uprint(x): print(f"{(x).n:.5f}“) # print number print(f"{(x).s:.5f}“) # print uncertainty
numbers = list(range(1, 17)) mean = statistics.mean(numbers) std = statistics.pstdev(numbers) n = len(numbers) se = std / (n**0.5)
def exp_func(x, a, b): return a * np.exp(b * x)
bl = [# nth bounce, milliseconds ERROR += 5 ms due to thats the yeah whatever (1, 440), (2, 310), (3, 260), (4, 190), (5, 150), (6, 120),]
b2l = [# nth bounce, milliseconds (1, 420), (2, 320), (3, 240), (4, 190), (5, 140), (6, 120),]
blp = [# nth bounce, milliseconds (1, 500), (2, 460), (3, 440), (4, 420), (5, 370), (6, 350), (7, 340), (8, 300), (9, 280), (10, 280), (11, 260), (12, 230),]
b2lp = [# nth bounce, milliseconds (1, 500), (2, 480), (3, 460), (4, 410), (5, 380), (6, 350), (7, 330), (8, 320), (9, 280), (10, 260), (11, 240), (12, 230),] tennisbounces1 = [ufloat(x[1], 5) / 1000 for x in bl] tennisbounces2 = [ufloat(x[1], 5) / 1000 for x in b2l] pingpongbounces1 = [ufloat(x[1], 5) / 1000 for x in blp] pingpongbounces2 = [ufloat(x[1], 5) / 1000 for x in b2lp]
def dofunction(list, name, what): plt.errorbar([x for x in range(len(list))], [math.log(x.n) for x in list], yerr=[x.s / x.n for x in list], capsize=5, color=“black”, ecolor=“red”,) plt.title(“Bounce time vs #Bounce” + what) plt.xlabel(“Bounce Number”) plt.ylabel(“ln(T) (ms)”) plt.savefig(name, dpi=300) plt.clf()

```
x = np.array([i + 1 for i in range(len(list))])
y = np.array([math.log(b.n) for b in list])
err_y = np.array([5 / x.n for x in list])
print(y)

x1 = sm.add_constant(x)
w = 1.0 / (err_y**2)
model = sm.WLS(y, x1, weights=w)
result = model.fit()

slope = result.params[1]
slope_error = result.bse[1]
slope_real = np.exp(slope)
slope_error_real = np.exp(slope) * slope_error

intercept = result.params[0]
intercept_error = result.bse[0]
intercept_real = np.exp(intercept)
intercept_error_real = np.exp(intercept) * intercept_error

print(f"intercept = {intercept_real:.4f} ± {intercept_error_real:.4f}")
```

```
print(f"slope = {slope_real:.4f} ± {slope_error_real:.4f}")

result.summary()
return [
    ufloat(slope_real, slope_error_real),
    ufloat(intercept_real, intercept_error_real),
]
```

def compsigma(a, b): return abs((a.n - b.n) / ((((a.s**2) + (b.s**2)) ** (1 / 2))))
a = dofunction(tennisbounces1, “tenboun1”, “on a tennis ball”) b = dofunction(tennisbounces2, “tenboun2”, “on a tennis ball”) c = dofunction(pingpongbounces1, “pp1”, “on a pingpong ball”) d = dofunction(pingpongbounces2, “pp2”, “on a pingpong ball”)
print(compsigma(a[0], b[0])) print(compsigma(c[0], d[0]))
def getv0(x: ufloat): return 9.81 * (x) / 2
print(ufloat(math.sqrt(2 * 9.81 * 0.4), 0)) print(getv0(a[1])) print(getv0(b[1])) print(getv0(c[1])) print(getv0(d[1])) v0 = (ufloat(math.sqrt(2 * 9.81 * 0.4), 0)) print(compsigma((getv0(a[1])), v0)) print(compsigma((getv0(b[1])), v0)) print(compsigma((getv0(c[1])), v0)) print(compsigma((getv0(d[1])), v0))